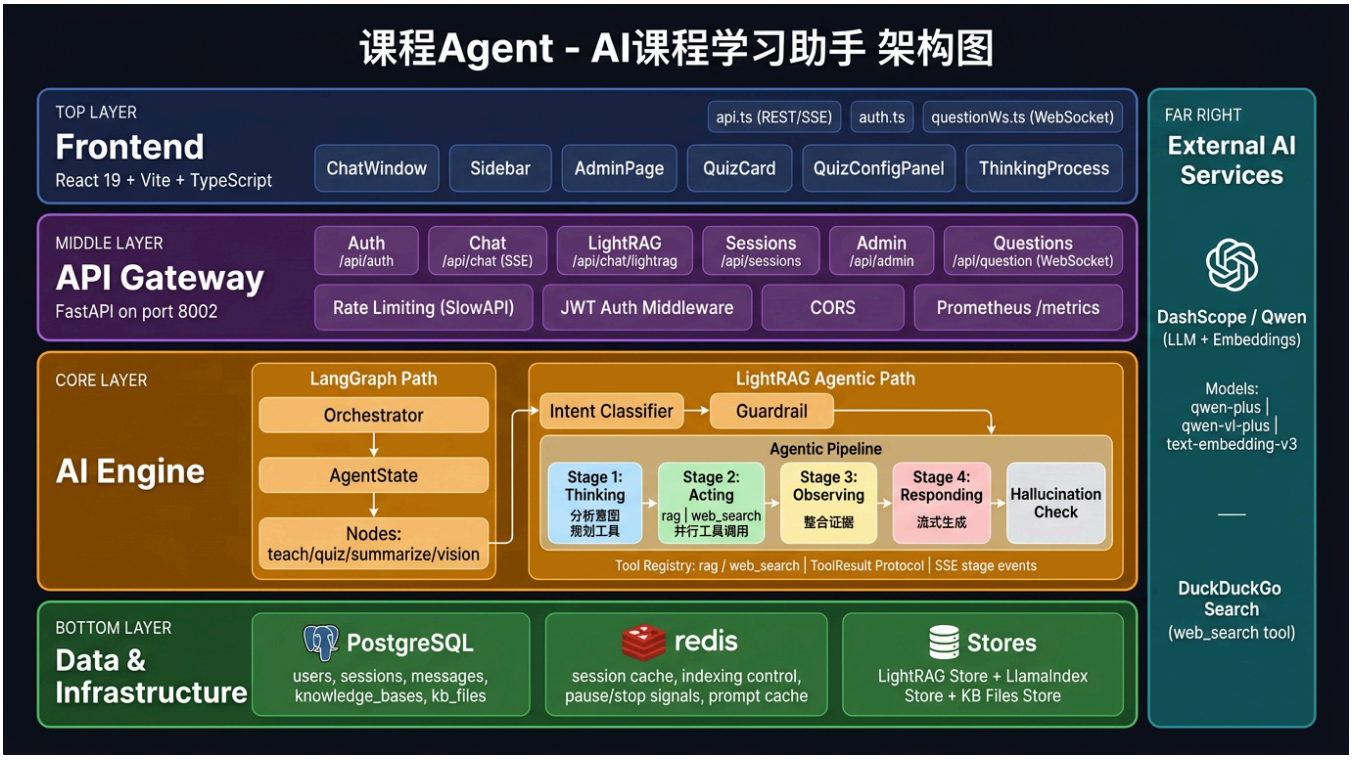
课程Agent — AI 课程学习助手 项目文档

> 本文档记录项目当前架构、核心功能、技术亮点及设计决策，供开发维护参考。



目录

1. 项目简介
2. 整体架构图
3. 技术栈
4. 目录结构
5. 后端架构详解
   - 5.1 API 层
   - 5.2 核心引擎层
   - 5.3 数据层
6. 前端架构详解
7. 核心功能说明
8. 亮点与技术改进
9. 数据流走向
10. 部署与配置
11. Agent 架构说明
12. 效果展示

---

1. 项目简介

课程Agent 是一套面向高校课程教学的 **AI 智能学习助手系统。**教师可在管理后台上传课程文档（PDF、Word、PPT），系统自动将文档解析入知识图谱；学生可通过聊天界面以自然语言提问，获得基于课程内容的精准回答，并支持 AI 自动出题与测验。

核心定位

- **课程专属知识库**：每门课程拥有独立知识图谱，AI 回答严格锚定课程内容
- **多模式智能对话**：支持教学讲解（teach）、答疑（quiz）、内容总结（summarize）、图片理解（vision）等多种模式
- **自动出题**：基于知识库内容，AI 协作多智能体自动生成多种题型
- **管理员后台**：可视化管理知识库上传、索引进度监控、用户管理

---

2. 整体架构图

架构图文件：`PROJECT_ARCHITECTURE_DIAGRAM.png`（见仓库根目录）

系统分为四层：

```
┌──────────────────────────────────────────────────────────────┐
│ 前端层（React 19 + Vite + TypeScript，浏览器 SPA）
│ ChatWindow / Sidebar / AdminPage / QuizCard / QuestionGenerator
└──────────────────────────────────────────────────────────────┘
                              │  HTTP / SSE / WebSocket
                              ▼
┌──────────────────────────────────────────────────────────────┐
│ API 网关层（FastAPI，端口 8002）
│ /api/chat  /api/chat/lightrag  /api/sessions  /api/admin
│ /api/auth  /api/question(WS)  /api/memory  /api/upload
│ SlowAPI 限流 | JWT 鉴权 | CORS | Prometheus /metrics
└──────────────────────────────────────────────────────────────┘
                              │
                ┌─────────────┴─────────────┐
                ▼                           ▼
┌──────────────────────────┐   ┌──────────────────────────┐
│ LangGraph 路径           │   │ LightRAG Agentic 路径    │
│ Orchestrator / AgentState │   │ 意图分类 → 护栏 → Pipeline │
│ 多模式节点（teach/quiz…）  │   │ Think→Act→Observe→Reply  │
│                          │   │ 工具：rag | web_search   │
│                          │   │ → 幻觉评估               │
└──────────────────────────┘   └──────────────────────────┘
                │                           │
                └─────────────┬─────────────┘
                              ▼
┌──────────────────────────────────────────────────────────────┐
│ 数据与基础设施层
│ PostgreSQL（用户/会话/消息/知识库元数据）
│ Redis（缓存、索引进度控制信号）
│ LightRAG Store / LlamaIndex Store / KB 文件目录
└──────────────────────────────────────────────────────────────┘
```

```
│ DashScope·通义千问（外部 LLM/嵌入，HTTP API）                │
└──────────────────────────────────────────────────────────┘
```

术语速查（括号内为简要含义，详细见后文技术栈与各章）：

缩写或词	说明
SSE	Server-Sent Events，服务端向浏览器单向推送文本事件（如聊天流式输出）。
WebSocket	全双工长连接，适合出题等需要服务端主动多帧推送的场景。
JWT	JSON Web Token，签名的访问令牌，用于无状态 API 鉴权。
CORS	Cross-Origin Resource Sharing，浏览器跨域访问后端时的安全策略头。
RAG	Retrieval-Augmented Generation，检索增强生成：先从知识库检索相关内容，再交给大模型组织答案。
LangGraph	基于有向图编排 LLM 与工具调用的框架（节点、边、状态机）。
LightRAG	图结构增强的 RAG 实现（实体/关系与向量结合检索），本项目用于构建课程知识库。
Agentic / Agent	智能体管线：模型可多步「思考—行动—观察—回复」，并调用工具（如 `rag`、`web_search`）。
ORM	Object-Relational Mapping，对象关系映射（如 SQLAlchemy 把表映射成 Python 类）。
护栏	对输入/输出做安全与合规检查（如敏感内容过滤），降低滥用与风险。
幻觉评估	对模型回答与检索依据的一致性做检查，减轻「编造未提供信息」的问题。
asyncpg	PostgreSQL 的异步驱动库，与 asyncio 配合，避免阻塞事件循环。
KB	Knowledge Base，知识库：课程文档及其索引/存储的逻辑集合。

---

3. 技术栈

层次	技术选型	说明
前端	React 19, Vite 8, TypeScript, Tailwind CSS 4	现代化 SPA，组件化开发
前端渲染	react-markdown, KaTeX	Markdown + 数学公式渲染
后端框架	FastAPI, Uvicorn, Gunicorn	异步高性能 Python Web 框架
AI 编排	LangGraph, LangChain	状态机驱动的多步 AI 工作流
知识图谱 RAG	lightrag-hku	图结构增强的 RAG 检索引擎
文档解析	LlamaIndex, LlamaParse, pypdf, python-docx, pymupdf	多格式文档解析与分块
向量检索	ChromaDB / FS（可选）	传统向量检索备选路径

层次	技术选型	说明
LLM / 嵌入	通义千问 (DashScope), qwen-plus/qwen-vl-plus/text-embedding-v3	国产大模型，OpenAI 兼容接口
数据库	PostgreSQL 16 + asyncpg + SQLAlchemy 2	异步 ORM，结构化数据存储
缓存/消息	Redis 7	会话缓存、索引进度控制信号
数据库迁移	Alembic	版本化 schema 管理
鉴权	PyJWT + bcrypt	JWT Token 认证 + 密码哈希
限流	SlowAPI	接口请求频率控制
可观测性	prometheus-fastapi-instrumentator	`/metrics` 端点暴露 Prometheus 指标
部署	Docker Compose, Dockerfile	容器化一键部署

---

4. 目录结构

```
课程agent/
├── backend/
│   ├── main.py                 # FastAPI 应用入口，路由挂载
│   ├── config.py               # 全局配置（环境变量读取）
│   ├── api/                    # 各业务路由模块
│   │   ├── auth.py             # 注册/登录/JWT
│   │   ├── chat.py             # LangGraph SSE 聊天
│   │   ├── lightrag.py         # LightRAG 图谱聊天
│   │   ├── sessions.py         # 会话 CRUD
│   │   ├── courses.py          # 课程列表
│   │   ├── upload.py           # 文件上传
│   │   ├── memory.py           # 用户记忆管理
│   │   ├── admin.py            # 管理员后台
│   │   ├── llama_rag.py        # LlamaIndex 建库/检索
│   │   ├── question.py         # 出题 WebSocket
│   │   └── sse.py              # 示例 SSE 端点
│   ├── core/                   # 核心业务逻辑
│   │   ├── orchestrator.py     # LangGraph 状态机编排
│   │   ├── lightrag_engine.py  # LightRAG 引擎 + agentic_pipeline
│   │   ├── prompts.py          # 所有 LLM prompt 集中管理（含 Agent 各阶段 prompt）
│   │   ├── tool_protocol.py    # ToolResult 数据协议
│   │   ├── tool_registry.py    # 工具实现注册表（rag / web_search）
│   │   ├── ingestion.py        # 文档解析与知识库摄入
│   │   ├── rag.py              # 传统 RAG 路径
│   │   ├── llm.py              # LLM 客户端（通义千问）
│   │   ├── memory.py           # 会话/消息 DB 操作
│   │   ├── learner_profile.py  # 学习者画像与记忆更新
│   │   ├── safety_pipeline.py  # 内容安全护栏
│   │   ├── database.py         # SQLAlchemy 模型与初始化
│   │   └── question/
│   │       └── coordinator.py  # 多智能体出题协调器
│   ├── kb_store/               # 知识库原始文件（按课程）
│   ├── lightrag_store/         # LightRAG 图谱工作区（按课程）
│   ├── knowledge/              # 内置 Markdown 课程材料
```

```
│   ├── uploads/                    # 用户上传文件
│   ├── alembic/                    # 数据库迁移脚本
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.tsx                 # 主壳：登录/管理/聊天状态切换
│   │   ├── main.tsx                # React 入口
│   │   ├── components/             # UI 组件
│   │   │   ├── ChatWindow.tsx      # 聊天主窗口（SSE 流式）
│   │   │   ├── Sidebar.tsx         # 课程/会话侧边栏
│   │   │   ├── MessageBubble.tsx   # 消息气泡
│   │   │   ├── AdminPage.tsx       # 管理后台页面
│   │   │   ├── QuizCard.tsx        # 题目展示卡片
│   │   │   ├── QuizConfigPanel.tsx# 测验配置面板
│   │   │   └ ...
│   │   ├── services/
│   │   │   ├── api.ts              # REST/SSE API 封装
│   │   │   ├── auth.ts             # 认证状态管理
│   │   │   └── questionWs.ts       # 出题 WebSocket 客户端
│   │   └── types/index.ts          # 全局类型定义
│   ├── vite.config.ts              # Vite 配置（API 代理）
│   └── package.json
└── docker-compose.yml              # 全服务编排
```

---

5. 后端架构详解

5.1 API 层

所有路由挂载在 `backend/main.py` ，统一前缀 `/api` 。

认证模块 `api/auth.py`

- `POST /api/auth/register` — 注册（bcrypt 密码哈希）
- `POST /api/auth/login` — 登录，返回 JWT Token（有效期 72 小时，可配置）
- `GET /api/auth/me` — 获取当前用户信息

聊天模块（两条路径）

LangGraph 路径 `api/chat.py`
- `POST /api/chat` — Server-Sent Events 流式聊天
- 底层调用 `core/orchestrator.py` 的 `run_agent_stream`
- 支持 teach / quiz / summarize / vision 四种模式节点

LightRAG 路径 `api/lightrag.py`
- `POST /api/chat/lightrag` — 图谱增强的 SSE 流式聊天
- 完整管线：意图分类 → 内容护栏 → **Agentic Pipeline（Thinking/Acting/Observing/Responding）** → 幻觉评估
- 前端通过 SSE `stage` / `stage_chunk` 事件可实时展示 Agent 推理过程
- `POST /api/chat/lightrag/index` — 触发某课程的 LightRAG 索引构建

会话管理 `api/sessions.py`

- `GET/POST /api/sessions` — 获取/创建会话
- `GET/PATCH/DELETE /api/sessions/{id}` — 获取/更新/删除会话
- `PATCH /api/sessions/{id}/mode` — 切换会话模式
- `GET/POST /api/sessions/{id}/messages` — 消息列表/追加消息

课程模块 `api/courses.py`

- `GET /api/courses` — 返回所有可见的知识库课程列表（仅 `is_visible=true`）

上传模块 `api/upload.py`

- `POST /api/upload` — 上传图片等附件，返回文件路径

记忆模块 `api/memory.py`

- `GET/PUT /api/memory` — 读写用户 profile 和 summary 记忆
- `POST /api/memory/refresh` — 基于对话历史重新生成记忆摘要
- `POST /api/memory/clear` — 清空用户记忆

管理员模块 `api/admin.py` + `api/llama_rag.py`

- 系统信息、知识库 CRUD
- 上传文件到知识库、触发/暂停/终止索引任务
- 索引进度实时查询（Redis 控制信号）
- 用户列表管理
- LlamaIndex 建库与独立检索接口

出题模块 `api/question.py`

- `WebSocket /api/question/generate` — 实时出题，多智能体协作，逐步推送生成进度

5.2 核心引擎层

LangGraph 编排器 `core/orchestrator.py`

- 定义 `AgentState` 状态模型，包含消息历史、模式、课程上下文等字段
- 路由节点根据 `mode`（teach / quiz / summarize / vision）分发到不同处理节点
- `normalize_mode` 函数处理模式字符串规范化
- 每个节点均可调用 RAG 检索丰富上下文

LightRAG 引擎 + Agentic Pipeline `core/lightrag_engine.py`

- 封装 lightrag-hku 库，管理按 `course_id` 隔离的工作区
- 使用 DashScope text-embedding-v3（1024 维）向量化
- 支持 `mix` 检索模式（图谱 + 向量融合）
- `is_lightrag_available()` 做可用性检查（需 `LIGHTRAG_ENABLED`、API Key、依赖安装）
- Top-K 检索可配置，默认 20 个文本块
- `agentic_pipeline`：四阶段 Agent 推理链

- **Thinking**：内部推理分析用户意图与工具规划，不直接展示给学生
- **Acting**：并行执行用户选定的工具（rag / web_search），收集工具 traces
- **Observing**：整合 Thinking + Tool Traces，提炼核心证据与回答要点
- **Responding**：基于观察结果流式生成面向学生的最终回答
- 每阶段均 yield `stage` / `stage_chunk` / `token` 事件，前端可实时展示推理过程

工具协议与注册表 `core/tool_protocol.py` / `core/tool_registry.py`

- `ToolResult`：统一工具返回数据结构（content / sources / success）
- `TOOLS_OPENAI_SCHEMA`：OpenAI function-calling 格式的工具 schema
- `execute_tool(name, course_id, **kwargs)`：统一工具调度入口
- 内置工具：
- **rag**：调用 LightRAG 知识图谱检索，`only_need_context=True` 模式，结果截断至 6000 字
- **web_search**：DuckDuckGo 免费搜索，返回 Top-5 结果

Prompt **集中管理** `core/prompts.py`

- 各阶段 Agent prompt 集中维护：`THINKING_PROMPT`、`OBSERVING_PROMPT`、`RESPONDING_PROMPT`、`ACTING_SYSTEM_PROMPT`、`ACTING_REACT_PROMPT`
- 课程 system_prompt 从数据库读取，Redis 缓存（TTL 10 分钟），`invalidate_course_prompt_cache` 支持实时失效

知识库摄入 `core/ingestion.py`

- `parse_files`：使用 LlamaIndex 解析 PDF/DOCX/PPT/TXT，自动分块
- 扫描件/图像 PDF 可走 LlamaParse（云端 OCR 解析，第 155–173 行）
- `ingest_to_lightrag`：批量写入 LightRAG 图谱
- `IndexingControl`：Redis 信号控制，支持**暂停**和**停止**索引任务
- 进度信息实时写入数据库 `knowledge_bases.progress`

LLM **客户端** `core/llm.py`

- 对接通义千问 DashScope，使用 OpenAI SDK 兼容接口
- 支持流式（`stream=True`）和非流式调用
- 文本模型（qwen-plus）、视觉模型（qwen-vl-plus）按需切换

学习者画像 `core/learner_profile.py`

- 每轮对话结束后异步更新 `users.summary_memory` 和 `users.profile_memory`
- 记忆内容注入后续对话的系统提示，实现跨会话个性化

安全护栏 `core/safety_pipeline.py`

- `classify_intent`：意图分类，区分课程学习问题 vs 无关/有害请求
- `evaluate_guardrail`：护栏判断，过滤不适当请求
- `evaluate_hallucination`：幻觉检测，评估回答与检索内容的一致性
- 在 LightRAG 路径（`api/lightrag.py`）完整使用

多智能体出题 `core/question/coordinator.py`

- `AgentCoordinator` 协调 Idea Agent、Generator Agent 等多个专职子智能体
- 支持工具标志：web_search（网络搜索）、rag（知识库检索）、code_execution（代码执行）
- 通过 WebSocket 实时推送生成进度

5.3 数据层

PostgreSQL 数据模型（`core/database.py`）

users 表

```
id, username, password_hash, display_name,
summary_memory, profile_memory,   ← AI 生成的学习者记忆
is_admin, created_at
```

sessions 表

```
id, course_id, user_id, title, mode, created_at, updated_at
```

messages 表

```
id, session_id(FK), role, content, msg_type, metadata_, created_at
```

knowledge_bases 表

```
id, course_id, name, description, is_visible,
status(pending/indexing/ready/error),
progress, chunks_total, chunks_done,   ← 索引进度追踪
lightrag_enabled, created_at, updated_at
```

kb_files 表

```
id, kb_id(FK), filename, file_path, file_size, status, created_at
```

初始化策略

- `init_db` 使用 PostgreSQL advisory lock 防止并发初始化冲突
- 采用 `create_all` + 运行时 `ALTER TABLE` 补齐列，而非完全依赖 Alembic

---

6. 前端架构详解

前端为无路由库的 SPA，`App.tsx` 通过 React state 在三种视图间切换：
1. **登录页** `LoginPage.tsx`
2. **管理后台** `AdminPage.tsx` （管理员专属）
3. **主聊天界面**（Sidebar + ChatWindow 组合）

关键组件

组件	功能
`ChatWindow.tsx`	聊天主窗口，发消息、接收 SSE 流式回答、模式切换
`Sidebar.tsx`	左侧边栏：课程列表、会话历史、登出、管理员入口
`MessageBubble.tsx`	消息气泡：支持 Markdown 渲染（含代码高亮、数学公式）
`AdminPage.tsx`	管理后台：知识库创建、文件上传、索引触发与进度监控
`QuizCard.tsx`	测验题目展示卡片（选择题/判断题/填空题）
`QuizConfigPanel.tsx`	测验配置：题型、数量、难度设置
`QuestionGeneratorPanel.tsx`	出题面板：WebSocket 连接，实时展示生成进度
`ThinkingProcess.tsx`	展示 AI 的推理思考过程（透明化 AI）
`SourceCard.tsx`	知识来源引用卡片（显示检索到的文档片段）

API 服务层

- `services/api.ts` ：根据 `ragEnabled` 自动选择 `/api/chat` （LangGraph）或 `/api/chat/lightrag` （LightRAG），对调用者透明
- `services/auth.ts`：JWT Token 本地存储与请求头注入
- `services/questionWs.ts`：出题 WebSocket 客户端封装

开发代理

`vite.config.ts`： `/api` 和 `/uploads` 代理到 `http://127.0.0.1:8002`，SSE 设置 `timeout: 0` （不超时）。

自动轮询

`App.tsx` 第 55–64 行：知识库处于 `indexing` 状态时，前端每 5 秒自动刷新课程列表，索引完成后实时更新 UI。

---

7. 核心功能说明

7.1 多模式 AI 对话

学生可在侧边栏或对话框切换以下模式：

- **讲解模式（teach）**：AI 以教师身份详细讲解知识点，结合课程文档
- **问答模式（quiz）**：AI 回答问题时附带知识点来源引用
- **总结模式（summarize）**：对某章节或对话内容进行结构化总结
- **视觉模式（vision）**：上传图片后，AI 结合图片内容（电路图、公式截图等）解答

7.2 课程知识库与 LightRAG

每门课程拥有独立的 LightRAG 图谱工作区（`lightrag_store/course_{id}/`）：

- 文档被解析为文本块，LightRAG 自动抽取实体和关系构建知识图谱
- 检索时使用 `mix` 模式（图谱遍历 + 向量相似度融合）
- 支持复杂关联查询（如"电路分析中的戴维南定理与诺顿定理的关系"）

7.3 智能出题系统

通过 WebSocket 实时生成题目：
1. 用户指定课程、题型、知识点范围
2. Idea Agent 生成出题思路
3. Generator Agent 根据知识库生成具体题目
4. 实时推送生成进度与题目到前端

支持的工具：
- **RAG 检索**：从课程知识库获取出题素材
- **网络搜索**：补充最新知识（可开关）
- **代码执行**：生成编程题时验证答案（可开关）

7.4 用户记忆与个性化

- 每次对话结束后，AI 自动更新用户的学习记忆（`summary_memory`）和学习画像（`profile_memory`）
- 这些记忆注入后续对话的系统提示，让 AI "记住"每位学生的学习进度和薄弱点
- 支持手动刷新或清空记忆

7.5 管理员后台

- 创建/编辑/删除知识库（课程）
- 批量上传文档（PDF/DOCX/PPT/TXT，最大 50MB）
- 一键触发 LightRAG 索引，实时显示进度（百分比 + 块数）
- 支持**暂停/继续**和**停止**索引任务（Redis 信号控制）
- 用户列表与权限管理

---

8. 亮点与技术改进

亮点 1：双路 RAG 架构，按需切换

系统同时维护两条 RAG 路径：

特性	LangGraph + 传统 RAG	LightRAG 图谱 RAG
检索精度	向量相似度	图谱遍历 + 向量融合
关联推理	弱	强（实体-关系图）
适用场景	简单事实查询	复杂关联问题

特性	LangGraph + 传统 RAG	LightRAG 图谱 RAG
启用方式	默认	`LIGHTRAG_ENABLED=true`

前端 `api.ts` 根据配置自动路由，对用户透明。

亮点 2：四阶段 Agentic Pipeline（ReAct 风格）

LightRAG 路径升级为真正的 Agent 架构，每次回答经历四个阶段：

阶段	作用	面向用户
Thinking	LLM 自主分析意图、规划工具	可在前端透明展示
Acting	并行调用 rag / web_search 工具	实时显示「检索中...」
Observing	整合工具结果提炼核心证据	内部阶段
Responding	流式生成面向学生的最终回答	主回答区渲染

工具通过 `core/tool_registry.py` + `core/tool_protocol.py` 统一注册和调度，ToolResult 协议保证扩展性。

亮点 3：完整的 AI 安全管线

LightRAG 路径在生成前后各有检查：
1. **意图分类**（`classify_intent`）：区分课程学习 vs 无关/有害请求
2. **内容护栏**（`evaluate_guardrail`）：过滤不当内容
3. **幻觉检测**（`evaluate_hallucination`）：对比检索内容与生成内容的一致性，将置信度评分附在响应元数据中

亮点 4：可中断的索引任务

知识库索引基于 Redis 信号实现暂停/恢复/停止：
- `IndexingControl` 在每批处理前检查 Redis 中的控制信号
- 管理员可随时暂停索引（如服务器负载过高），继续时从断点恢复
- 避免了大文件索引时的不可控长时间阻塞

亮点 5：学习者记忆持久化

区别于普通聊天机器人，系统维护两种记忆：
- `summary_memory`：本次会话的内容摘要（由 AI 自动生成）
- `profile_memory`：用户的长期学习画像（知识薄弱点、已掌握内容等）

这两种记忆跨会话保留，注入每次对话的上下文，使 AI 真正"了解"学生。

亮点 6：生产级工程实践

- **异步全链路**：FastAPI + asyncpg + SQLAlchemy 2 异步 ORM，无阻塞 I/O
- **数据库安全初始化**：PostgreSQL advisory lock 防止多 worker 并发 `init_db` 的竞态
- **限流防刷**：SlowAPI 对聊天、上传等关键接口设置请求频率限制

- **可观测性**：Prometheus `/metrics` 端点，开箱即用接入 Grafana
- **4 worker Gunicorn**：生产环境多进程，充分利用多核

亮点 7：多格式文档支持

- PDF（含扫描件，通过 LlamaParse OCR）
- Word (.docx)
- PowerPoint (.pptx)
- 纯文本 (.txt, .md)
- 图像 PDF走视觉模型路径

---

9. 数据流走向

学生聊天请求（LightRAG Agentic 路径）

```
浏览器 ChatWindow
  → [SSE] POST /api/chat/lightrag              api/lightrag.py:71
  → JWT 验证 + 限流检查
  → classify_intent()                          core/safety_pipeline.py
      ↓ 课程相关
  → evaluate_guardrail()                       core/safety_pipeline.py
      ↓ 通过
  → agentic_pipeline(course_id, message, ...)  core/lightrag_engine.py:510
      ├── [Stage 1] Thinking                   → SSE stage_chunk
      │     LLM 内部推理：分析意图、规划工具
      ├── [Stage 2] Acting (并行)              → SSE stage_chunk
      │     rag tool  → core/tool_registry.py  → LightRAG 检索
      │     web_search tool → DuckDuckGo
      ├── [Stage 3] Observing                  → SSE stage_chunk
      │     LLM 整合 Thinking + Tool Traces → 提炼证据
      └── [Stage 4] Responding (流式)           → SSE token
            LLM 基于观察结果面向学生生成最终回答
  → evaluate_hallucination()                   core/safety_pipeline.py
  → update_learner_memory()                    core/learner_profile.py
  → 保存消息到 PostgreSQL                      core/memory.py
```

管理员上传并索引知识库

```
AdminPage.tsx
  → POST /api/admin/kb/{course_id}/upload   api/admin.py
  → 文件写入 kb_store/{course_id}/raw/
  → POST /api/admin/kb/{course_id}/index    api/admin.py
  → 后台任务 _run_indexing()
  → parse_files() LlamaIndex 解析分块       core/ingestion.py:176
  → ingest_to_lightrag() 批量写入图谱        core/ingestion.py:220
      ↓ 每批前检查 Redis 控制信号
  → 实时更新 knowledge_bases.progress
  → 前端 5 秒轮询，显示进度百分比
```

出题请求

```
QuestionGeneratorPanel.tsx
  → WebSocket /api/question/generate         api/question.py:28
  → AgentCoordinator.generate_from_topic()   core/question/coordinator.py
  → Idea Agent 生成出题思路
  → Generator Agent 调用 RAG 检索素材
  → 生成题目 JSON
  → WebSocket 实时推送 → QuizCard 渲染
```

---

10. 部署与配置

环境变量（关键项）

变量	默认值	说明
`DASHSCOPE_API_KEY`	必填	通义千问 API 密钥
`DATABASE_URL`	`postgresql+asyncpg://postgres:postgres@localhost:5432/course_agent`	PostgreSQL 连接
`REDIS_URL`	`redis://localhost:6379/0`	Redis 连接
`JWT_SECRET`	dev-secret	JWT 签名密钥
`LIGHTRAG_ENABLED`	`false`	是否启用 LightRAG
`TEXT_MODEL`	`qwen-plus`	文本生成模型
`VISION_MODEL`	`qwen-vl-plus`	视觉理解模型
`EMBEDDING_MODEL`	`text-embedding-v3`	向量嵌入模型
`LLAMA_CLOUD_API_KEY`	可选	LlamaParse 扫描件解析

本地开发启动

```
# 后端
cd backend
cp .env.example .env    # 填写 DASHSCOPE_API_KEY 等
pip install -r requirements.txt
uvicorn main:app --reload --port 8002

# 前端
cd frontend
npm install
npm run dev             # 默认 5173，代理到 8002
```

Docker Compose 部署

```
docker compose up --build
# 前端: http://localhost:80
# 后端: http://localhost:8002
# 指标: http://localhost:8002/metrics
```

Docker Compose 包含：
- `postgres:16`（数据持久化挂载到 `./data/postgres`）
- `redis:7`
- `backend`（Python 3.12-slim，Gunicorn 4 workers）
- `frontend`（Nginx 静态服务）

默认内置课程

系统启动时自动写入以下内置课程（`core/_builtin_courses.py`）：
- 邮票知识（stamp）
- 电路分析基础（circuit）

---

11. Agent 架构说明

- **自主推理（Thinking）**：LLM 主动分析问题、决定是否及如何使用工具，而非被动执行固定流程
- **工具使用（Acting）**：Agent 可调用外部工具（rag、web_search），动态补充知识
- **证据整合（Observing）**：Agent 自我反思工具返回结果，类似 ReAct 中的 Observation 步骤
- **受控生成（Responding）**：基于完整推理链生成最终输出，与普通 RAG 直接生成不同

与普通 RAG 的区别

维度	普通 RAG	Agentic Pipeline
检索触发	每次固定检索	LLM 自主决定是否检索
工具数量	单一（向量检索）	可扩展（rag + web_search + 未来更多）
推理透明度	不可见	Thinking/Observing 可向前端暴露
并行能力	无	Acting 阶段多工具并行

关键文件位置

文件	作用
`backend/core/lightrag_engine.py`	`agentic_pipeline` 四阶段实现
`backend/core/prompts.py`	Thinking / Observing / Responding / Acting prompt
`backend/core/tool_registry.py`	rag / web_search 工具实现
`backend/core/tool_protocol.py`	`ToolResult` 统一协议

文件	作用
`backend/api/lightrag.py`	SSE 入口，将 pipeline 事件流式推送前端

12. 效果展示